Práctica 1.3 — Organización con APIRouter

Objetivo: separar las rutas por recurso.

Enunciado: crea `app/routers/tasks.py` y `app/routers/projects.py` con `APIRouter(prefix="/tasks", tags=["tasks"])` y análogo; mueve los endpoints existentes a su router; inclúyelos con `app.include_router(...)`; verifica en Swagger que aparecen agrupados por tag.

Criterios de aceptación: no quedan rutas de negocio en `main.py`. Swagger muestra los grupos "tasks" y "projects".

Entregable: estructura de carpetas `routers/` + captura de Swagger agrupado + acceso al repositorio GitHub.

Criterio	NL	EP	L	E
Uso de APIRouter	Sin routers; todo en `main.py`	Routers creados pero sin usar	Routers por recurso con `prefix` y `tags`	Routers versionados (`/api/v1/...`) y `__init__.py` en el paquete
Separación de responsabilidades	`main.py` mezcla todo	Separación parcial	`main.py` solo instancia y configura; la lógica vive en los routers	Estructura completa `routers/`, `schemas/`, `models/`, `core/`
Registro de routers	Routers no incluidos	`include_router` sin prefijos coherentes	`include_router` correcto y Swagger agrupado por tags	Documentación Swagger enriquecida por grupo
Prefijos y convención de URLs	Rutas planas y confusas	Prefijos parciales	`/tasks`, `/projects` con convención REST clara	Nombres de recursos en plural consistente en toda la API
Concepto de modularidad	No entiende por qué separar	Distinción parcial	Explica el beneficio de separar rutas por recurso	Compara el coste de mantener un monolito frente a un diseño modular
Documentación	Sin estructura documentada	Notas parciales	Estructura de carpetas documentada	Justificación de la estructura elegida